Spring Security Guide. Part 2

🆚 JWT vs Token opaque (différences simples)

Token Opaque	JWT (JSON Web Token)
Juste une chaîne aléatoire (UUID)	Contient des infos utiles (encodées)
Stocké côté serveur (en mémoire, Redis…)	Stocké et vérifié directement par le token lui-même
Aucune info visible dedans	Contient un **header**, un **payload** et une **signature**
Nécessite une base de données ou cache pour valider	Peut être validé sans accéder à la base ou au cache

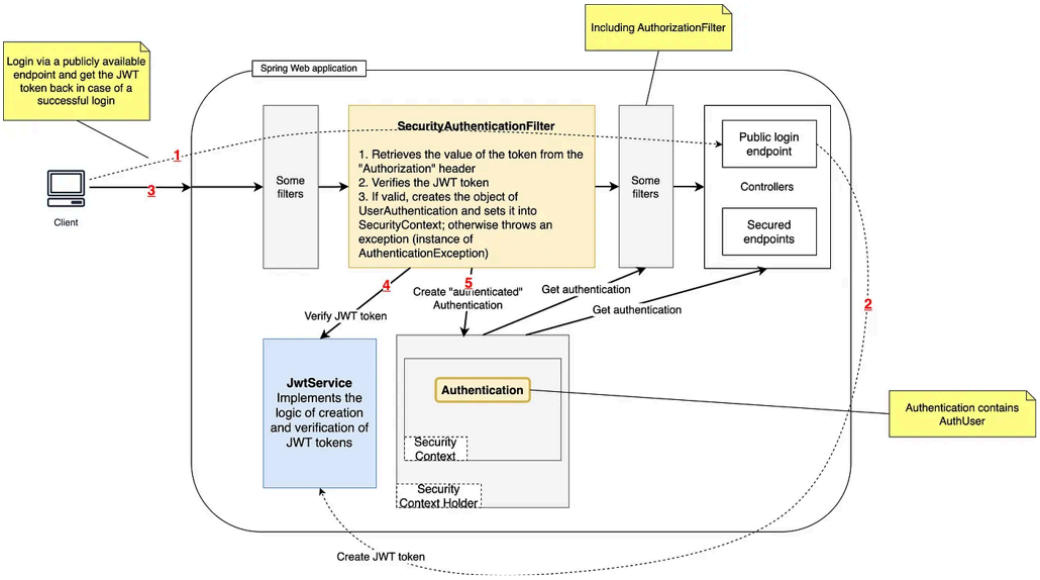
Il contient 3 parties, séparées par des points :

1. **Header** : infos sur le type de token et la méthode de signature (encodé en base64)
2. **Payload** : les **données** (userId, rôles, etc.) (aussi encodé en base64)
3. **Signature** : pour **vérifier** que le token n'a pas été modifié

🔐 Signature : deux méthodes possibles

1. **Symétrique (HMAC)**
   → 1 seule **clé secrète partagée** pour signer **et** vérifier le token.
   → Plus simple, mais moins sécurisé si la clé fuit.
2. **Asymétrique (RSA / EC)**
   → 2 clés : une **privée** (pour signer) et une **publique** (pour vérifier).
   → Plus sécurisé, utilisé dans les gros systèmes.

✅ Dans ton exemple, on utilise **symétrique** (plus simple).



🧹 On supprime (part-1) `AuthUserCache`

❌ **Pourquoi ?**

Parce que :

- Avec **JWT**, les données de l'utilisateur (id, rôles…) sont **déjà dans le token**.
- On **n'a plus besoin de stocker** les tokens côté serveur.
- Le système devient **stateless** (sans état côté backend).

✅ On crée une classe `JwtService`

Cette classe aura 2 responsabilités :

1. `createToken(AuthUser)`

→ Génère un JWT signé (contenant les infos de l'utilisateur).

2. `verifyToken(token)`

→ Vérifie la **signature** du token.

→ Si OK, **extrait les infos** (id, rôles…) pour construire un `AuthUser`.

🔐 Pourquoi on peut faire confiance au token ?

- Le token est **signé avec une clé secrète**.
- Si quelqu'un **modifie** le contenu → la **signature ne correspond plus** → rejeté.
- Donc on peut **se fier aux infos** dans le token **sans les vérifier dans la base**.

🧰 Quelle librairie utiliser ?

Dans l'exemple :

```
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.0.0</version>
</dependency>
```

```
@Service
public class JwtService {


 private static final String ROLES_CLAIM = "roles";


 private final Algorithm signingAlgorithm;


 public JwtService(@Value("${jwt.signing-secret}") String signingSecret) {


   // this example uses a symmetric signature of the JWT token, but if you want the issuer and the
   // verifier of the JWT token to be different applications you may want to use an asymmetric
   // signature


   this.signingAlgorithm = Algorithm.HMAC256(signingSecret);
 }


 public AuthUser resolveJwtToken(String token) {
   try {
     JWTVerifier verifier = JWT.require(signingAlgorithm).build();
     DecodedJWT decodedJWT = verifier.verify(token);


     String userId = decodedJWT.getSubject();
     List<Role> roles = decodedJWT.getClaim(ROLES_CLAIM).asList(Role.class);


     return new AuthUser(userId, roles);
   } catch (JWTVerificationException exception) {
     throw new TokenAuthenticationException("JWT is not valid");
   }
 }


 public String createJwtToken(AuthUser authUser) {
   long nowMillis = System.currentTimeMillis();
   Date now = new Date(nowMillis);
   long expMillis = nowMillis + 3600000; // 1 hour validity
   Date exp = new Date(expMillis);


   List<String> roles = authUser.roles().stream().map(Role::name).toList();


   return JWT.create()
       .withSubject(authUser.userId())
```

```
        .withClaim(ROLES_CLAIM, roles)
        .withIssuedAt(now)
        .withExpiresAt(exp)
        .sign(signingAlgorithm);
 }
}
```

🔐 2. Définir la clé secrète dans `application.yaml`

```yaml
jwt:
  signing-secret: some-secret
```

⚠️ **En production :**

- ❌ Ne jamais mettre la clé en dur dans le code ou dans le YAML !
- ✅ Utilise une **variable d'environnement** ou un **config server** sécurisé.

🚫 3. Plus besoin de méthode `logout`

- Avec JWT : **"se déconnecter" = supprimer le token côté client** (navigateur, mobile, etc.).
- Si tu veux forcer la déconnexion (**invalider un token avant sa fin de validité**), tu dois :
  → Créer une **blacklist** (liste noire de tokens).
  → Ce n'est **pas fait ici**, car ce tuto reste simple.

🔄 4. Adapter la méthode `login`

Tu ne stockes plus le token, tu **le crées avec** `JwtService` :

```java
@Service
public class AuthService {


 private final UserService userService;


 private final PasswordEncoder passwordEncoder;


 private final JwtService jwtService;


 public AuthService(
     UserService userService, PasswordEncoder passwordEncoder, JwtService jwtService) {
   this.userService = userService;
   this.passwordEncoder = passwordEncoder;
   this.jwtService = jwtService;
 }


 public TokenDto login(LoginDto loginDto) {


```

```java
    UserResponseWithCredentials userCredentials =
        userService.getUserCredentialsByUsername(loginDto.username());


    if (!passwordEncoder.matches(loginDto.password(), userCredentials.passwordHash())) {
      throw new ApplicationAuthenticationException("Password is incorrect");
    }


    UserResponse userResponse = userCredentials.userResponse();
    AuthUser authUser = new AuthUser(userResponse.id(), userResponse.roles());


    String jwtToken = jwtService.createJwtToken(authUser);


    return new TokenDto(jwtToken);
  }
}
```

🔁 5. Adapter le filtre de sécurité

Avant, tu allais chercher l'utilisateur via `AuthUserCache`.

Maintenant, tu **valides et décodes** le token JWT via `JwtService` :

```java
@Component
public class SecurityAuthenticationFilter extends OncePerRequestFilter {

  private final JwtService jwtService;

  public SecurityAuthenticationFilter(JwtService jwtService) {
    this.jwtService = jwtService;
  }

  @Override
  protected void doFilterInternal(
      HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
      throws ServletException, IOException {

    String authenticationHeader = request.getHeader(AuthConstants.AUTHORIZATION_HEADER);

    if (authenticationHeader == null) {
      // Authentication token is not present, let's rely on anonymous authentication
      filterChain.doFilter(request, response);
      return;
    }

    String jwtToken = stripBearerPrefix(authenticationHeader);
    AuthUser authUser = jwtService.resolveJwtToken(jwtToken);

    UserAuthentication authentication = new UserAuthentication(authUser);

    SecurityContext securityContext = SecurityContextHolder.createEmptyContext();
    securityContext.setAuthentication(authentication);
    SecurityContextHolder.setContext(securityContext);

```

```
    filterChain.doFilter(request, response);
  }

  String stripBearerPrefix(String token) {

    if (!token.startsWith("Bearer")) {
      throw new TokenAuthenticationException("Unsupported authentication scheme");
    }

    return token.substring(7);
  }
}
```

Changement	Avant (Opaque Token)	Maintenant (JWT)
Stockage token	`AuthUserCache` (Map ou Redis)	❌ Aucun stockage
Authentification	Lire le token + chercher dans la cache	Lire et **valider le token signé**
Déconnexion	Supprimer de la cache	Juste **oublier** le token côté client
Sécurité	Pas de sens dans le token	Données **encodées + signées**
Clé secrète	Pas utilisée	🔐 Requise pour signer/valider le JWT